Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues

ASSIGNMENT REPORT

Submitted in the partial fulfilment for the Course

of

CSA0305 - Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

Artificial Intelligence and Data Science

CS & BS

Submitted by

Shahana (192524479)

C. Mokshitha Reddy (192571054)

Under the Supervision of

Dr.Uma Priyadarsini P.S

Saveetha School of Engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

A. Assignment Information

Particular	Details
Department	Department of Computer Science and Engineering
Programme	Bachelor of Engineering in Computer Science Engineering
Course Code & Course Name	CSA0305 – Data Structures
Academic Year / Batch	2026–2027
Faculty Name	DR. UMA PRIYADARSINI
Assignment Title	Design an Intelligent Disaster-Relief Supply Chain Using Graphs, Heaps and Priority Queues
Date of Issue	01/09/2026
Date of Submission	02/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply suitable Abstract Data Type for construction of the disaster-relief distribution network. CO4: Use priority queues and heaps for dynamic delivery prioritization and route caching for repeated relief queries in C. CO5: Develop a robust disaster-relief supply chain system using graph-based prioritization and optimal distribution-planning algorithms.
Bloom's Taxonomy Level	L4/L5 – Analyze / Evaluate
SDG Mapping	SDG 1 – No Poverty; SDG 2 – Zero Hunger; SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action
Industry / Societal Relevance	Intelligent emergency logistics for prioritizing urgent relief requests, selecting feasible routes and using limited transport and supply resources efficiently.

B. Assignment Problem / Challenge

A disaster-relief agency must allocate food, medicine, water and shelter kits from multiple warehouses to multiple disaster-affected zones. Each zone may have a different demand, urgency and distance from available warehouses. Transportation routes have variable distances and capacities, while vehicles and relief resources are limited. Requests can arrive dynamically and urgency may increase as a disaster worsens.

The challenge is to design a complete C-based distribution system that represents the warehouse-to-zone network as a weighted graph and uses a Min-Heap or Max-Heap priority queue to dynamically rank and

dispatch deliveries according to a computed priority score. At least two prioritization strategies must be evaluated, and the final strategy should balance speed, fairness and resource utilization.

C. Problem Statement

The proposed system processes a dynamic stream of disaster-relief requests with minimal avoidable delay. It must identify which affected zone should receive relief next, select an appropriate warehouse and feasible route, and ensure that transport and resource capacities are respected. The system must also support priority changes when new information is received and should avoid indefinite starvation of lower-priority zones.

The student develops a complete C program integrating a weighted graph, a heap-based priority queue and a route selection method. The implementation is followed by test cases, execution evidence, performance observations and a comparison of alternative prioritization strategies.

D. Requirements and Constraints Functional Requirements

- Read and store warehouse, disaster-zone and route details. Represent the relief network as a weighted graph.
- Store demand and urgency information for affected zones.
- Calculate a priority score for active delivery requests.
- Insert and remove requests using a heap-based priority queue.
- Select a feasible warehouse and route for delivery.
- Respect transport, vehicle and available-resource constraints.
- Support dynamic re-prioritization when requests or conditions change. Report the resulting distribution plan.

Performance Requirements

- Heap insertion and deletion should operate in O(log n).
- Heap top/peek should operate in O(1).
- Graph traversal should be efficient for the chosen representation.
- Shortest-path processing should use an appropriate efficient algorithm for non-negative route costs.
- Repeated route queries should avoid unnecessary recomputation where route caching is implemented.
- System behaviour should be measured for increasing workloads where applicable.

Technical Constraints

- Implementation language: C.
- Graph representation: [ADJACENCY LIST].
- Priority queue: [MIN-HEAP / MAX-HEAP].
- Route algorithm: [DIJKSTRA].
- Performance measurement: [clock()].

Reliability, Safety and Ethical Constraints

- Relief allocations must not exceed available supply or route/vehicle capacity.
- Priority updates must not corrupt the queue ordering.
- Invalid routes or unavailable resources should be handled safely.
- Emergency decisions should remain explainable through the recorded priority factors. Real beneficiary data should only be used for legitimate relief operations.

E. Student Work 1. Problem Understanding and Formulation

The problem is to build a data-structure-driven disaster-relief logistics engine. A simple first-come-firstserved queue cannot adequately represent changing emergency conditions. A graph is appropriate because warehouses, zones and transportation links naturally form a network. A heap-based priority queue is appropriate because the system repeatedly needs access to the currently highest-priority feasible request.

Expected Outcomes

- Correct representation of the warehouse-to-zone distribution network.
- Dynamic priority ranking of disaster requests.
- Efficient highest-priority request retrieval using a heap.
- Feasible route selection under distance and capacity constraints.
- Dynamic re-prioritization when urgency or demand changes.
- Comparison of alternative prioritization strategies.
- Evidence-based selection of a final distribution strategy.

Available Data

Field	Example
Warehouse	W1
Disaster Zone	Z1
Demand	80 units
Urgency	95 / 100
Route Distance	12 km
Route Capacity	50 units
Resource Type	Medicine / Food / Water / Shelter Kit
Vehicle Capacity	100 units

Assumptions

- Urgency and demand are represented by numeric values suitable for normalization.
- Route distances are non-negative.
- Each route has a known capacity or the implementation explicitly documents if capacity is modeled at vehicle level.
- Warehouse stock is represented by the quantities available to the prototype.
- The priority formula weights are fixed for an experiment and may be varied for comparison.
- Fairness is represented through a waiting-time or starvation-prevention component if implemented.

2. Application of Course Knowledge

Data Structure / Technique	Application	Complexity
Weighted Graph	Warehouse, zone and route representation	O(V + E) storage with adjacency lists
Adjacency List	Stores outgoing routes and their costs/capacities	O(V + E)
Priority Queue	Maintains pending relief requests by priority	O(log n) insertion/deletion
Heap	Implements the priority queue	O(1) top; O(log n) update/delete

Shortest-Path Algorithm	Finds a low-cost feasible route	Typically O((V+E)logV) with a binary heap
Route Cache [if implemented]	Stores repeated route-query results	Depends on cache structure
Structures / Records	Stores zone, warehouse and request attributes	O(1) field access

3. Solution / Design / Methodology

The system follows a sequential but dynamically updateable relief-dispatch pipeline. Network data are loaded first. Each disaster zone is represented as a delivery request containing demand, urgency and other relevant attributes. The system calculates a priority score and inserts the request into a heap-based priority queue. When dispatching, the top request is checked for feasibility before resources are committed.

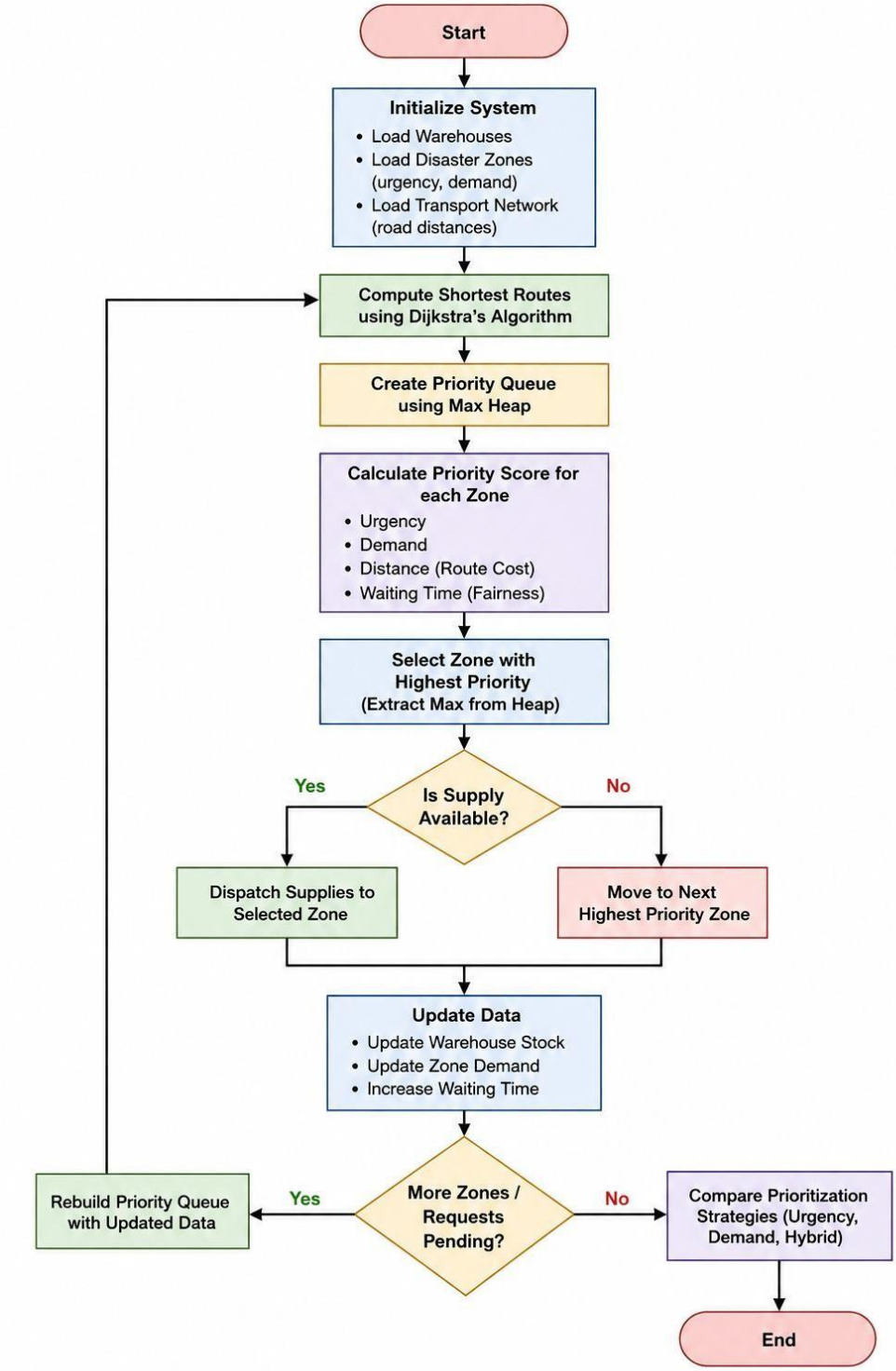
A representative hybrid priority formulation is:

$\mathbf{P_i = w_u U_i + w\_dD_i + w\_fF_i - w\_rR_i}$ where U is urgency, D is normalized demand, F is a fairness/waiting factor, R represents route cost or distance, and the w values are experiment-specific weights. The final source code must state the exact formula and weights used.

Algorithm

1. Initialize the weighted graph, warehouse inventory and disaster-zone records.
2. Read each relief request and calculate its priority score.
3. Insert each eligible request into the heap-based priority queue.
4. Extract the current highest-priority request.
5. Check warehouse stock, vehicle capacity and route feasibility.
6. Select an appropriate route using the implemented route-selection method.
7. Dispatch the feasible quantity and update all relevant state.
8. If conditions change, update demand/urgency and re-prioritize affected requests.
9. Continue until the stopping condition is reached.

FIG. 1 FLOW CHART OF DISASTER-RELIEF SUPPLY CHAIN ENGINE



4. Use of Modern Tools

Tool	Purpose
C / GCC	Program development and compilation
Visual Studio Code / [IDE]	Source-code editing and execution
GitHub	Version control and submission
Terminal	Compilation and execution
clock() / [MEASUREMENT METHOD]	Execution-time measurement

The implementation can be compiled using GCC and tested from a terminal or the selected IDE. Screenshots of the source code, compilation, execution and output should be added as evidence of tool usage.

5. Implementation Code

```
/*
============================================================
 CSA0305 - DATA STRUCTURES - SLOT A

 Assignment:
 Design an Intelligent Disaster-Relief Supply Chain
 Using Graphs, Heaps and Priority Queues

 Features:
1. Weighted Graph using Adjacency List
2. Dijkstra's Shortest Path Algorithm
3. Max Heap / Priority Queue
4. Urgency-First Strategy
5. Demand-First Strategy
6. Hybrid Priority Strategy
7. Fairness / Waiting-Time Handling
8. Dynamic Re-prioritization
9. Warehouse Stock Management
10. Vehicle Capacity Constraint
11. Strategy Comparison
12. Menu Driven Interface

 Compile:
 gcc main.c -o disaster_relief

 Run:
 ./disaster_relief
============================================================
*/

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <limits.h>

#define MAX_NODES 20
#define MAX_WAREHOUSES 5
#define MAX_ZONES 10
```

```
#define MAX_HEAP 50
#define INF 999999

/* ============================================================
   DATA STRUCTURES
   ============================================================ */

/* Graph node */ typedef
struct GraphNode {
    int vertex;
int distance;
int capacity;
    struct GraphNode *next;
} GraphNode;

/* Warehouse */
typedef struct {
int id;
    char name[50];
int stock;     int x;
int y;
} Warehouse;

/* Disaster Zone */
typedef struct {
int id;
    char name[50];

    int urgency;          /* 1 - 10 */     int demand;
/* percentage / demand units */     int population;
    int requiredSupply;
int deliveredSupply;
    int waitingTime;

} DisasterZone;

/* Delivery Request */
typedef struct {

    int warehouseId;
    int zoneId;

    int distance;
    int routeCapacity;
```

```
    float priority;

} DeliveryRequest;

/* Max Heap */
typedef struct {

    DeliveryRequest heap[MAX_HEAP];
int size;

} MaxHeap;


/* ============================================================
   GLOBAL VARIABLES
   ============================================================ */

GraphNode *graph[MAX_NODES];

Warehouse warehouses[MAX_WAREHOUSES];
DisasterZone zones[MAX_ZONES];

int warehouseCount = 0; int
zoneCount = 0;
int nodeCount = 0;


/* ============================================================
   UTILITY FUNCTIONS
   ============================================================ */

void clearInputBuffer()
{
    int c;

    while ((c = getchar()) != '\n' && c != EOF)
        ;
}


/* ============================================================
   GRAPH FUNCTIONS
   ============================================================ */
```

```
/*
 Add one directed edge.
*/
void addEdge(int source, int destination,
int distance, int capacity)
{
   GraphNode *newNode =
      (GraphNode *)malloc(sizeof(GraphNode));

   if (newNode == NULL)
   {
      printf("Memory allocation failed.\n");
exit(1);
   }

   newNode->vertex = destination;     newNode-
>distance = distance;
   newNode->capacity = capacity;

   newNode->next = graph[source];

   graph[source] = newNode;
}


/*
 Add an undirected route.
*/
void addRoute(int source, int destination,
int distance, int capacity)
{     addEdge(source, destination, distance,
capacity);     addEdge(destination, source, distance,
capacity);
}


/*
 Initialize graph.
*/
void initializeGraph()
{
   int i;
```

```
    for (i = 0; i < MAX_NODES; i++)
        graph[i] = NULL;
}


/*
 Display graph.
*/ void
displayNetwork()
{
    int i;

    printf("\n");
    printf("================================================\n");
printf("            TRANSPORT NETWORK\n");
printf("================================================\n");

    for (i = 0; i < nodeCount; i++)
    {
        GraphNode *current = graph[i];

        printf("Node %d -> ", i);

        while (current != NULL)
        {
printf(
                "[Node %d | %d km | Capacity %d] -> ",
                current->vertex,                current-
>distance,
                current->capacity
            );

            current = current->next;
        }

        printf("NULL\n");
    }

    printf("================================================\n");
}


/* ============================================================
   DIJKSTRA'S SHORTEST PATH ALGORITHM
```

```
  ========================================================== */

void dijkstra(int source, int distances[])
{
   int visited[MAX_NODES];
    int
i;
   int count;

   for (i = 0; i < nodeCount; i++)
   {
      distances[i] = INF;
visited[i] = 0;
   }

   distances[source] = 0;

   for (count = 0; count < nodeCount; count++)
   {
      int u = -1;
      int minimum = INF;

      /*
       Find the unvisited node
with minimum distance.
      */
      for (i = 0; i < nodeCount; i++)
      {           if
(!visited[i] &&
            distances[i] < minimum)
         {
            minimum = distances[i];
            u = i;
         }
      }

      if (u == -1)
         break;

      visited[u] = 1;

      /*
       Relax adjacent edges.
      */
```

```
        GraphNode *current = graph[u];

        while (current != NULL)
        {
            int v = current->vertex;

            if (!visited[v] &&
distances[u] != INF &&
distances[u] + current->distance
                    < distances[v])
            {
distances[v] =
distances[u] +
current->distance;
            }

            current = current->next;
        }
    }
}


/*
 Get route capacity between two nodes.
*/
int getRouteCapacity(int source, int destination)
{
    GraphNode *current = graph[source];

    while (current != NULL)
    {
        if (current->vertex == destination)
            return current->capacity;

        current = current->next;
    }

    return 0;
}


/* ============================================================
   MAX HEAP FUNCTIONS
   ============================================================ */
```

```
void swapRequests(DeliveryRequest *a,
                  DeliveryRequest *b)
{
   DeliveryRequest temp = *a;

   *a = *b;
   *b = temp;
}


/*
 Initialize heap.
*/
void initializeHeap(MaxHeap *heap)
{     heap->size =
0;
}


/*
 Insert request into Max Heap.
*/
void insertHeap(MaxHeap *heap,
                DeliveryRequest request)
{     int
index;     int
parent;

   if (heap->size >= MAX_HEAP)
   {
      printf("Priority queue is full.\n");
      return;
   }

   index = heap->size;

   heap->heap[index] = request;

   heap->size++;

   while (index > 0)
   {
      parent = (index - 1) / 2;
```

```
        if (heap->heap[parent].priority >=
            heap->heap[index].priority)
        {
break;
        }

        swapRequests(
            &heap->heap[parent],
             &heap->heap[index]
        );

        index = parent;
    }
}



/*
 Remove highest-priority request.
*/
DeliveryRequest extractMax(MaxHeap *heap)
{
    DeliveryRequest result;

    result.zoneId = -1;

    if (heap->size <= 0)
        return result;

    result = heap->heap[0];

    heap->size--;

    if (heap->size > 0)         heap-
>heap[0] =
            heap->heap[heap->size];

    int index = 0;

    while (1)
    {
        int left = 2 * index + 1;
int right = 2 * index + 2;
        int largest = index;
```

```
        if (left < heap->size &&            heap-
>heap[left].priority >
            heap->heap[largest].priority)
        {
            largest = left;
        }

        if (right < heap->size &&           heap-
>heap[right].priority >
            heap->heap[largest].priority)
        {
            largest = right;
        }

        if (largest == index)
            break;

        swapRequests(           &heap-
>heap[index],
            &heap->heap[largest]
        );

        index = largest;
    }

    return result;
}


/*
 Display heap.
*/
void displayHeap(MaxHeap *heap)
{
    int i;

    printf("\n");
    printf("================================================\n");
printf("            PRIORITY QUEUE\n");
    printf("================================================\n");

    for (i = 0; i < heap->size; i++)
```

```
    {
printf(
            "%d. Zone %d | Priority %.3f | Distance %d km\n", i
            + 1,
            heap->heap[i].zoneId,           heap-
>heap[i].priority,          heap->heap[i].distance
        );
    }

    printf("==================================================\n");
}


/* ============================================================
   PRIORITY STRATEGIES
   ============================================================ */

/*
 Normalize demand to 0 - 1.
*/
float normalizeDemand(int demand)
{     if (demand >
100)
        demand = 100;

    return demand / 100.0f;
}


/*
 Normalize urgency to 0 - 1.
*/ float normalizeUrgency(int
urgency)
{
    if (urgency > 10)
        urgency = 10;

    if (urgency < 0)
        urgency = 0;

    return urgency / 10.0f;
}
```

```
/*  Distance
score.

 Shorter distance gets higher score.
*/
float distanceScore(int distance)
{     if (distance <=
0)
       return 1.0f;

   return 1.0f / (1.0f + distance);
}


/*  Strategy
1:
 Urgency First */
float urgencyFirst(int urgency)
{
   return normalizeUrgency(urgency);
}


/*  Strategy
2:
 Demand First
*/
float demandFirst(int demand)
{
   return normalizeDemand(demand);
}


/*  Strategy
3:
 Hybrid strategy.

 50% urgency
30% demand
 20% distance
*/ float hybridPriority(int
urgency,                  int
demand,
                  int distance)
```

```
{     float u =
normalizeUrgency(urgency);     float d =
normalizeDemand(demand);
   float dist = distanceScore(distance);

   return
      (0.50f * u) +
      (0.30f * d) +
      (0.20f * dist);
}



/*
 Fairness-aware hybrid strategy.

 Waiting time increases priority gradually.
*/
float fairHybridPriority(int urgency,
                         int demand,
int distance,                      int
waitingTime)
{     float base =
hybridPriority(
urgency,
demand,
distance
      );

   float fairnessBonus =
      waitingTime * 0.02f;

   return base + fairnessBonus;
}



/* ============================================================    FIND BEST
WAREHOUSE
  ============================================================ */

int findNearestWarehouse(int zoneNode,
int distances[][MAX_NODES])
{     int bestWarehouse = -
1;
   int bestDistance = INF;
```

```
    int
i;

    for (i = 0; i < warehouseCount; i++)
    {
        int d = distances[i][zoneNode];

        if (d < bestDistance &&
            warehouses[i].stock > 0)
        {
            bestDistance = d;
            bestWarehouse = i;
        }
    }

    return bestWarehouse;
}


/* ============================================================
   BUILD PRIORITY QUEUE
   ============================================================ */

void buildPriorityQueue(
MaxHeap *heap,
    int strategy)
{
    int distances[MAX_WAREHOUSES][MAX_NODES];
    int
i;    int
j;

    initializeHeap(heap);

    /*
     Run Dijkstra from every warehouse.
    */
    for (i = 0; i < warehouseCount; i++)
    {        dijkstra(
warehouses[i].id,
            distances[i]
        );
    }
```

```
   /*
    Create one request for each zone.
   */
   for (i = 0; i < zoneCount; i++)
   {
      int zoneNode = zones[i].id;

      int warehouse =
findNearestWarehouse(
zoneNode,              distances
         );

      if (warehouse == -1)
         continue;

      int shortestDistance =
         distances[warehouse][zoneNode];

      int routeCapacity =
getRouteCapacity(
warehouses[warehouse].id,
zoneNode
         );

      DeliveryRequest request;

      request.warehouseId =
warehouse;       request.zoneId =
         i;

      request.distance =
         shortestDistance;

      request.routeCapacity =
         routeCapacity;

      /*
       Select priority strategy.
      */

      if (strategy == 1)
      {
request.priority =
```

```
urgencyFirst(
zones[i].urgency
            );
      }       else if
(strategy == 2)
      {
request.priority =
demandFirst(
zones[i].demand
            );       }       else
{           request.priority =
fairHybridPriority(
zones[i].urgency,
zones[i].demand,
shortestDistance,
               zones[i].waitingTime
            );
      }

      insertHeap(heap, request);
   }
}



/* ============================================================
  DISPLAY DISASTER ZONES
  ============================================================ */

void displayZones()
{
   int i;

   printf("\n");
   printf("=================================================================\n");
printf("                DISASTER ZONES\n");
   printf("=================================================================\n");

   printf(
      "%-5s %-15s %-10s %-10s %-12s %-12s %-10s\n",
      "ID",
      "Zone",
      "Urgency",
      "Demand",
      "Population",
```

```
        "Required",
        "Waiting"
    );

    printf("-------------------------------------------------------------\n");

    for (i = 0; i < zoneCount; i++)
    {
        printf(
            "%-5d %-15s %-10d %-10d %-12d %-12d %-10d\n",
zones[i].id,            zones[i].name,          zones[i].urgency,
zones[i].demand,        zones[i].population,
zones[i].requiredSupply,
            zones[i].waitingTime
        );
    }

    printf("=============================================================\n");
}


/* ============================================================
   DISPLAY WAREHOUSES
   ============================================================ */

void displayWarehouses()
{
    int i;

    printf("\n");
    printf("=============================================\n");
printf("            WAREHOUSES\n");
    printf("=============================================\n");

    for (i = 0; i < warehouseCount; i++)
    {
        printf(
            "Warehouse %d | %-15s | Stock: %d units\n",
warehouses[i].id,           warehouses[i].name,
            warehouses[i].stock
        );
    }

    printf("=============================================\n");
```

```
}


/* ============================================================
DISPATCH RELIEF
  ============================================================ */

void dispatchRelief(int strategy)
{
    MaxHeap heap;

    int vehicleCapacity;

    int totalDelivered = 0;
    int totalDistance = 0;

    int deliveries = 0;

    printf("\nEnter vehicle capacity: ");

    if (scanf("%d", &vehicleCapacity) != 1)
    {
        clearInputBuffer();
printf("Invalid input.\n");
        return;
    }

    if (vehicleCapacity <= 0)
    {
        printf("Vehicle capacity must be positive.\n");
return;
    }

    /*
     Build priority queue.
    */
    buildPriorityQueue(
        &heap,
        strategy
    );

    if (heap.size == 0)
    {
```

```
        printf("No delivery requests available.\n");
return;
    }

    printf("\n");
    printf("================================================================\n");
printf("                  RELIEF DISPATCH\n");
    printf("================================================================\n"); /*
     Process highest priority requests.
    */
    while (heap.size > 0)
    {
        DeliveryRequest request =
            extractMax(&heap);

        int zoneIndex =
            request.zoneId;

        int warehouseIndex =
            request.warehouseId;

        if (zoneIndex < 0)
            continue;

        /*
         Check warehouse stock.
        */
        if (warehouses[warehouseIndex].stock <= 0)
        {
            printf(
                "Warehouse %d has no stock.\n",
warehouseIndex
            );

            continue;
        }

        /*
         Determine quantity to deliver.
        */        int
quantity =
            zones[zoneIndex].requiredSupply;

        if (quantity > vehicleCapacity)
```

```
            quantity = vehicleCapacity;

        if (quantity >
warehouses[warehouseIndex].stock)        {
            quantity =
warehouses[warehouseIndex].stock;
        }

        /*
         Display dispatch.
        */
        printf("\n");
printf("Dispatch #%d\n",
               deliveries + 1);

        printf(
            "Warehouse : %s\n",
            warehouses[warehouseIndex].name
        );          printf(
"Zone      : %s\n",
            zones[zoneIndex].name
        );
        printf(
            "Urgency   : %d/10\n",
            zones[zoneIndex].urgency
        );
        printf(
"Demand    : %d\n",
            zones[zoneIndex].demand
        );
        printf(
            "Distance  : %d km\n",
            request.distance
        );          printf(
"Priority  : %.3f\n",
            request.priority
        );

printf(
            "Quantity  : %d units\n",
            quantity
        );

        /*
```

```
         Capacity warning.
        */        if
(quantity <
           zones[zoneIndex].requiredSupply)
        {
           printf(
              "Status    : PARTIAL DELIVERY\n"
           );
}         else
{
           printf(
              "Status    : FULL DELIVERY\n"
           );
        }

        /*
         Update stock.
        */
        warehouses[warehouseIndex].stock
           -= quantity;

        /*
         Update zone demand.
        */
        zones[zoneIndex].requiredSupply
           -= quantity;

        zones[zoneIndex].deliveredSupply
           += quantity;

        /*
         Update demand percentage.
        */
        if (zones[zoneIndex].requiredSupply <= 0)
        {
           zones[zoneIndex].demand = 0;
        }
else
        {
           zones[zoneIndex].demand -=
              quantity / 2;

           if (zones[zoneIndex].demand < 0)
              zones[zoneIndex].demand = 0;
```

```
        }

        totalDelivered += quantity;

        totalDistance +=
            request.distance;

        deliveries++;
    }

    printf("\n");
    printf("================================================================\n");
printf("                DISPATCH SUMMARY\n");
    printf("================================================================\n");

    printf(
        "Total Deliveries       : %d\n",
        deliveries
    );

    printf(
        "Total Supply Delivered : %d units\n",
        totalDelivered
    );

    printf(
        "Total Distance Travelled: %d km\n",
totalDistance
    );

    if (deliveries > 0)
    {
        printf(
            "Average Delivery Distance: %.2f km\n",
            (float)totalDistance /
            deliveries
        );
    }

    printf("================================================================\n");
}
```

```
/* ============================================================
DISPLAY PRIORITIES
  ============================================================ */

void displayPriorities()
{
   MaxHeap urgencyHeap;
   MaxHeap demandHeap;
   MaxHeap hybridHeap;

   int i;

   buildPriorityQueue(
      &urgencyHeap,
      1
   );

   buildPriorityQueue(
      &demandHeap,
      2
   );

   buildPriorityQueue(
      &hybridHeap,
      3
   );

   printf("\n");
printf("================================================================\n");
printf("                PRIORITY COMPARISON\n");
   printf("================================================================\n");

   printf("\nURGECY-FIRST STRATEGY\n");
   printf("-----------------------------\n");

   while (urgencyHeap.size > 0)
   {
      DeliveryRequest r =
         extractMax(&urgencyHeap);

      printf(
         "Zone %-3d Priority %.3f\n",
r.zoneId,
         r.priority
```

```
        );
    }

    printf("\nDEMAND-FIRST STRATEGY\n");
    printf("------------------------------\n");

    while (demandHeap.size > 0)
    {
        DeliveryRequest r =
            extractMax(&demandHeap);

        printf(
            "Zone %-3d Priority %.3f\n",
r.zoneId,
            r.priority
        );
    }

    printf("\nHYBRID + FAIRNESS STRATEGY\n");
    printf("------------------------------\n");

    while (hybridHeap.size > 0)
    {
        DeliveryRequest r =
extractMax(&hybridHeap);        printf(
            "Zone %-3d Priority %.3f\n",
r.zoneId,
            r.priority
        );
    }

    printf("============================================================\n");
}


/* ============================================================
SHORTEST ROUTES
   ============================================================ */

void displayShortestRoutes()
{
    int distances[MAX_NODES];

    int i;
```

```
   printf("\n");
   printf("================================================================\n");
printf("                SHORTEST ROUTES\n");
   printf("================================================================\n");

   for (i = 0; i < warehouseCount; i++)
   {       dijkstra(
warehouses[i].id,
          distances
       );

       printf("\nFrom %s:\n",
            warehouses[i].name);

       int j;

       for (j = 0; j < zoneCount; j++) {
          int zoneNode =
              zones[j].id;

          if (distances[zoneNode] == INF)
          {
              printf(
                  "  -> %s : No route\n",
zones[j].name
              );
}           else
{
              printf(
                  "  -> %-15s : %d km\n",
zones[j].name,
                  distances[zoneNode]
              );
          }
       }
   }

   printf("================================================================\n");
}


/* ============================================================
UPDATE URGENCY
```

```
   ========================================================== */

void updateZoneUrgency()
{     int
zone;
    int urgency;

    displayZones();

    printf("\nEnter Zone ID to update: ");

    if (scanf("%d", &zone) != 1)
    {
        clearInputBuffer();
        printf("Invalid input.\n");
        return;
    }

    if (zone < 0 || zone >= zoneCount)
    {
        printf("Invalid zone ID.\n");
        return;
    }

    printf(
        "Enter new urgency (1-10): "
    );

    if (scanf("%d", &urgency) != 1)
    {         clearInputBuffer();
printf("Invalid input.\n");
        return;
    }

    if (urgency < 1 || urgency > 10)
    {
        printf(
            "Urgency must be between 1 and 10.\n"
        );

return;
    }

    zones[zone].urgency =
```

```
        urgency;

    /*
     Increase waiting time slightly      to
demonstrate dynamic behaviour.
    */
    zones[zone].waitingTime = 0;

    printf(
        "\nUrgency successfully updated.\n"
    );

    printf(
        "Zone: %s\n",
        zones[zone].name
    );

    printf(
        "New urgency: %d/10\n",
        zones[zone].urgency
    );

    printf(
        "Priority queue will be rebuilt automatically.\n"
    );
}


/* ============================================================
   INCREASE WAITING TIME
   ============================================================ */

void increaseWaitingTime()
{
    int i;

    for (i = 0; i < zoneCount; i++)
    {
        if (zones[i].requiredSupply > 0)
        {
            zones[i].waitingTime++;
        }
    }
```

```
    printf(
        "\nWaiting time updated for pending zones.\n"
    );
}


/* ============================================================
PERFORMANCE COMPARISON
  ============================================================ */

void performanceComparison()
{
    int i;

    float urgencyTotal = 0;
float demandTotal = 0;
    float hybridTotal = 0;

    int urgencyHigh = 0;
int demandHigh = 0;
    int hybridHigh = 0;

    printf("\n");
    printf("================================================================\n");
printf("            PRIORITIZATION STRATEGY ANALYSIS\n");
printf("================================================================\n");

    for (i = 0; i < zoneCount; i++)
    {
        /*
         Example distance used for
strategy comparison.
        */

        int distance = 10 + i * 5;

        float u =
urgencyFirst(
              zones[i].urgency
           );

        float d =
demandFirst(
```

```
            zones[i].demand
        );

    float h =
        fairHybridPriority(
            zones[i].urgency,
zones[i].demand,            distance,
zones[i].waitingTime
        );

    urgencyTotal += u;
demandTotal += d;
    hybridTotal += h;

    if (zones[i].urgency >= 8)
    {           if (u
>= 0.8)
            urgencyHigh++;

        if (d >= 0.8)
            demandHigh++;

        if (h >= 0.6)
            hybridHigh++;
    }
  }

  printf("\n");
  printf(
    "%-25s %-15s %-15s\n",
    "Strategy",
    "Average Score",
    "High Urgency"
  );

  printf(
    "---------------------------------------------------------------\n"
  );

  printf(
    "%-25s %-15.3f %-15d\n",
"Urgency                First",
urgencyTotal   /   zoneCount,
urgencyHigh
```

```
    );

    printf(
        "%-25s %-15.3f %-15d\n",
        "Demand First",
demandTotal / zoneCount,
demandHigh
    );

    printf(
        "%-25s %-15.3f %-15d\n",
"Hybrid + Fairness",        hybridTotal
/ zoneCount,        hybridHigh
    );

    printf(
        "\nRecommended strategy: HYBRID + FAIRNESS\n"
    );

    printf(
        "Reason: It balances urgency, demand, distance\n"
        "and waiting time rather than optimizing only one factor.\n"
    );

    printf("============================================================\n");
}


/* ============================================================
   INITIALIZE SAMPLE DATA
   ============================================================ */

void initializeSampleData()
{
    /*
     ------------------------------------------------------
     WAREHOUSES
     Node IDs: 0, 1, 2
     ------------------------------------------------------
*/

    warehouseCount = 3;

    warehouses[0].id = 0;
```

```
    strcpy(
warehouses[0].name,
        "Warehouse-A"
    );

    warehouses[0].stock = 500;


    warehouses[1].id = 1;

    strcpy(
warehouses[1].name,
        "Warehouse-B"
    );

    warehouses[1].stock = 450;


    warehouses[2].id = 2;

    strcpy(
warehouses[2].name,
        "Warehouse-C"
    );

    warehouses[2].stock = 400;


    /*
     ----------------------------------------------------------
     DISASTER ZONES
     Node IDs: 3 - 8
     ----------------------------------------------------------
*/

    zoneCount = 6;

    zones[0].id = 3;

    strcpy(
zones[0].name,
        "Zone-Alpha"
    );
```

```
    zones[0].urgency = 9;
zones[0].demand = 90;
zones[0].population = 5000;
zones[0].requiredSupply = 180;
zones[0].deliveredSupply = 0;
    zones[0].waitingTime = 0;


    zones[1].id = 4;

    strcpy(
zones[1].name,
        "Zone-Beta"
    );

    zones[1].urgency = 7;
zones[1].demand = 75;
zones[1].population = 3500;
zones[1].requiredSupply = 140;
zones[1].deliveredSupply = 0;
    zones[1].waitingTime = 1;


    zones[2].id = 5;

    strcpy(
zones[2].name,
        "Zone-Gamma"
    );

    zones[2].urgency = 5;
zones[2].demand = 80;
zones[2].population = 4200;
zones[2].requiredSupply = 160;
zones[2].deliveredSupply = 0;
    zones[2].waitingTime = 2;


    zones[3].id = 6;

    strcpy(
zones[3].name,
        "Zone-Delta"
```

```
    );

    zones[3].urgency = 8;
zones[3].demand = 60;
zones[3].population = 2800;
zones[3].requiredSupply = 120;
zones[3].deliveredSupply = 0;
    zones[3].waitingTime = 3;



    zones[4].id = 7;

    strcpy(
zones[4].name,
        "Zone-Epsilon"
    );

    zones[4].urgency = 4;
zones[4].demand = 55;
zones[4].population = 2200;
zones[4].requiredSupply = 100;
zones[4].deliveredSupply = 0;
    zones[4].waitingTime = 4;



    zones[5].id = 8;

    strcpy(
zones[5].name,
        "Zone-Zeta"
    );

    zones[5].urgency = 10; zones[5].demand
    = 95;
    zones[5].population = 6000;
zones[5].requiredSupply = 200;
zones[5].deliveredSupply = 0;
    zones[5].waitingTime = 0;



    /*
     Total nodes:
     0,1,2 = warehouses
     3,4,5,6,7,8 = disaster zones
```

```
   */

   nodeCount = 9;


   /*
    ------------------------------------------------------------
    TRANSPORT NETWORK
    ------------------------------------------------------------

    addRoute(
source,
destination,
distance,
capacity
    );
   */

   addRoute(0, 1, 10, 200);
addRoute(0, 3, 15, 100);     addRoute(0,
4, 25, 100);

   addRoute(1, 2, 12, 150);
addRoute(1, 4, 10, 120);     addRoute(1,
5, 20, 100);

   addRoute(2, 5, 8, 100);
   addRoute(2, 6, 18, 100);

   addRoute(3, 4, 12, 80);
    addRoute(3, 6, 30, 60);

   addRoute(4, 5, 10, 80);
addRoute(4, 7, 20, 70);

   addRoute(5, 6, 12, 80);
addRoute(5, 8, 25, 70);

   addRoute(6, 7, 10, 60);
addRoute(6, 8, 15, 70);

   addRoute(7, 8, 10, 50);
}
```

```
/* ============================================================
RESET DATA
  ============================================================ */

void resetData()
{
   int i;
   /*
    Reset warehouse stock.
   */

   warehouses[0].stock = 500;
warehouses[1].stock = 450;     warehouses[2].stock
= 400;



   /*
    Reset zone supply.
   */

   int supply[] =
   {
      180,
      140,
      160,
      120,
      100,
      200
   };

   int demand[] =
   {
      90,
      75,
      80,
      60,
      55,
      95
   };

   for (i = 0; i < zoneCount; i++)
   {
      zones[i].requiredSupply =
```

```
        supply[i];

      zones[i].deliveredSupply =
        0;

      zones[i].demand =
demand[i];
   }

   printf(
      "\nSystem data reset successfully.\n"
   );
}



/* ============================================================
MENU
  ============================================================ */

void displayMenu()
{
   printf("\n\n");
printf("====================================================================\n");
printf("       INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN\n");
printf("====================================================================\n");

   printf("1. Display Warehouses\n");     printf("2.
Display Disaster Zones\n");     printf("3. Display
Transport Network\n");     printf("4. Find Shortest
Routes\n");     printf("5. Calculate Priority
Queue\n");     printf("6. Run Urgency-First
Dispatch\n");     printf("7. Run Demand-First
Dispatch\n");     printf("8. Run Hybrid + Fairness
Dispatch\n");     printf("9. Update Zone
Urgency\n");     printf("10. Increase Waiting
Time\n");     printf("11. Compare Prioritization
Strategies\n");     printf("12. Reset Simulation
Data\n");
   printf("0. Exit\n");

   printf("====================================================================\n");

   printf("Enter your choice: ");
}
```

```
/* ============================================================
MAIN FUNCTION
  ============================================================ */

int main()
{
   int choice;

   initializeGraph();

   initializeSampleData();

   printf("\n");
   printf("*****************************************************************\n");
printf("*                                               *\n");
   printf("*      INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN            *\n");
   printf("*                                               *\n");
   printf("*      Graph + Dijkstra + Max Heap + Priority Queue         *\n");
   printf("*                                               *\n");
   printf("*****************************************************************\n");

   do
   {
      displayMenu();

      if (scanf("%d", &choice) != 1)
      {
         clearInputBuffer();

         printf(
            "\nInvalid input. Please enter a number.\n"
         );

         continue;
      }

      switch (choice)
      {
case 1:

            displayWarehouses();

            break;
```

```
            case
2:

               displayZones();

               break;

            case
3:

               displayNetwork();

               break;

            case
4:

               displayShortestRoutes();

break;

            case
5:
           {
               MaxHeap heap;

               int strategy;
                printf("\n");
printf("Select strategy:\n");
printf("1. Urgency First\n");
printf("2. Demand First\n");
               printf("3. Hybrid + Fairness\n");

               printf(
                   "Enter strategy: "
               );

               scanf("%d", &strategy);

               if (strategy < 1 ||
strategy > 3)                {
printf(
                     "Invalid strategy.\n"
```

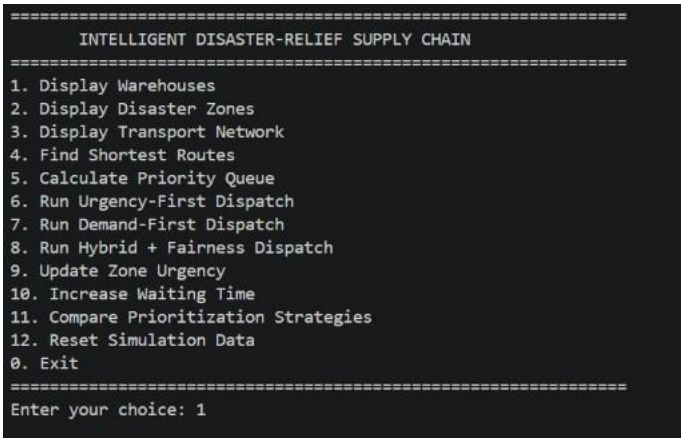
```
            );
            break;
        }

        buildPriorityQueue(
&heap,              strategy
        );

        displayHeap(&heap);

        break;
    }

    case
6:

        resetData();

        printf(
            "\nRunning Urgency-First Strategy...\n"
        );

        dispatchRelief(1);

        break;

    case
7:

        resetData();

        printf(
            "\nRunning Demand-First Strategy...\n"
        );

        dispatchRelief(2);

        break;

    case
8:

        resetData();
```

```
            printf(
                "\nRunning Hybrid + Fairness Strategy...\n"
            );

            dispatchRelief(3);

            break;

        case
9:

            updateZoneUrgency();

            break;

        case
10:

            increaseWaitingTime();

            break;

        case
11:

            performanceComparison();

            break;

        case
12:

resetData();

            break;

        case
0:

            printf(
                "\nExiting Disaster Relief System...\n"
            );
            printf(
                "Thank you.\n"
```

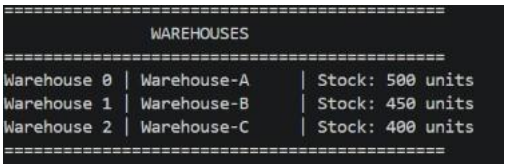
```
            );

            break;


default:

            printf(
                "\nInvalid choice. Please try again.\n"
            );
        }

    } while (choice != 0);


    /*
     Free graph memory.
    */

    for (int i = 0; i < nodeCount; i++)
    {
        GraphNode *current =
            graph[i];

        while (current != NULL)
        {
            GraphNode *temp =
                current;

            current =
                current->next;

            free(temp);
        }
    }

    return 0; }
```

The implementation should contain, at minimum, structures and functions for graph management, disaster-zone/request storage, priority-score calculation, heap operations, route selection and dispatch updates. If route caching is implemented, the cache structure and hit/miss behaviour should also be documented.

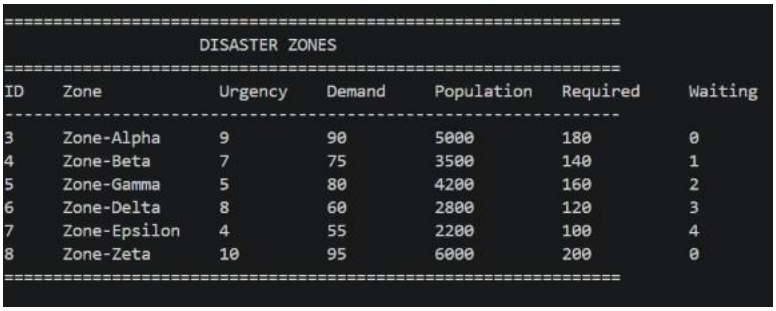
Implementation Evidence



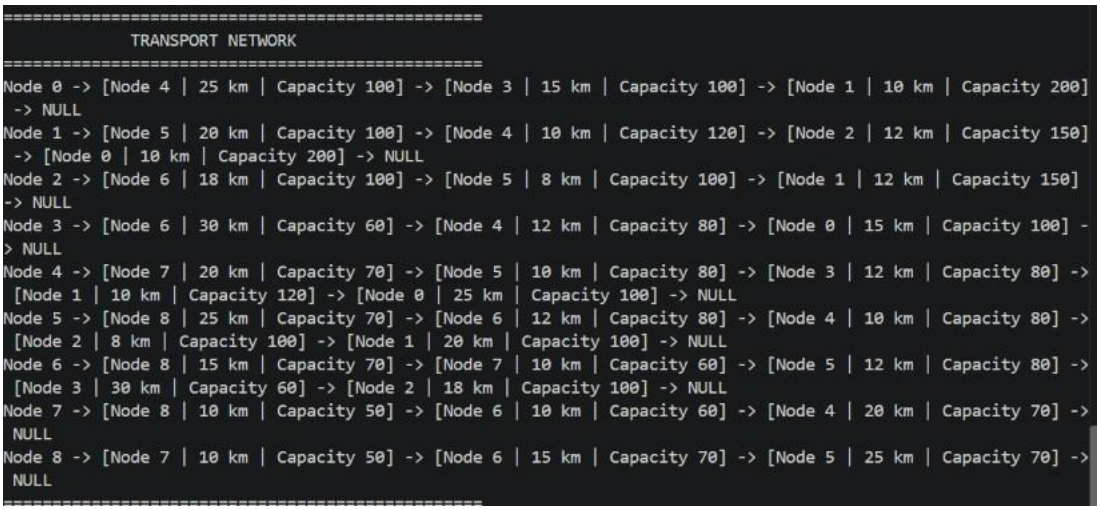
```
============================================================
       INTELLIGENT DISASTER-RELIEF SUPPLY CHAIN
============================================================
1. Display Warehouses
2. Display Disaster Zones
3. Display Transport Network
4. Find Shortest Routes
5. Calculate Priority Queue
6. Run Urgency-First Dispatch
7. Run Demand-First Dispatch
8. Run Hybrid + Fairness Dispatch
9. Update Zone Urgency
10. Increase Waiting Time
11. Compare Prioritization Strategies
12. Reset Simulation Data
0. Exit
============================================================
Enter your choice: 1
```



```
============================================
              WAREHOUSES
============================================
Warehouse 0 | Warehouse-A     | Stock: 500 units
Warehouse 1 | Warehouse-B     | Stock: 450 units
Warehouse 2 | Warehouse-C     | Stock: 400 units
============================================
```



```
============================================================
                    DISASTER ZONES
============================================================
ID    Zone           Urgency    Demand     Population   Required     Waiting
------------------------------------------------------------
3     Zone-Alpha     9          90         5000         180          0
4     Zone-Beta      7          75         3500         140          1
5     Zone-Gamma     5          80         4200         160          2
6     Zone-Delta     8          60         2800         120          3
7     Zone-Epsilon   4          55         2200         100          4
8     Zone-Zeta      10         95         6000         200          0
============================================================
```



```
================================================
            TRANSPORT NETWORK
================================================
Node 0 -> [Node 4 | 25 km | Capacity 100] -> [Node 3 | 15 km | Capacity 100] -> [Node 1 | 10 km | Capacity 200]
 -> NULL
Node 1 -> [Node 5 | 20 km | Capacity 100] -> [Node 4 | 10 km | Capacity 120] -> [Node 2 | 12 km | Capacity 150]
 -> [Node 0 | 10 km | Capacity 200] -> NULL
Node 2 -> [Node 6 | 18 km | Capacity 100] -> [Node 5 | 8 km | Capacity 100] -> [Node 1 | 12 km | Capacity 150]
-> NULL
Node 3 -> [Node 6 | 30 km | Capacity 60] -> [Node 4 | 12 km | Capacity 80] -> [Node 0 | 15 km | Capacity 100] -
> NULL
Node 4 -> [Node 7 | 20 km | Capacity 70] -> [Node 5 | 10 km | Capacity 80] -> [Node 3 | 12 km | Capacity 80] ->
 [Node 1 | 10 km | Capacity 120] -> [Node 0 | 25 km | Capacity 100] -> NULL
Node 5 -> [Node 8 | 25 km | Capacity 70] -> [Node 6 | 12 km | Capacity 80] -> [Node 4 | 10 km | Capacity 80] ->
 [Node 2 | 8 km | Capacity 100] -> [Node 1 | 20 km | Capacity 100] -> NULL
Node 6 -> [Node 8 | 15 km | Capacity 70] -> [Node 7 | 10 km | Capacity 60] -> [Node 5 | 12 km | Capacity 80] ->
 [Node 3 | 30 km | Capacity 60] -> [Node 2 | 18 km | Capacity 100] -> NULL
Node 7 -> [Node 8 | 10 km | Capacity 50] -> [Node 6 | 10 km | Capacity 60] -> [Node 4 | 20 km | Capacity 70] ->
 NULL
Node 8 -> [Node 7 | 10 km | Capacity 50] -> [Node 6 | 15 km | Capacity 70] -> [Node 5 | 25 km | Capacity 70] ->
 NULL
================================================
```

```
============================================================
                    SHORTEST ROUTES
============================================================

From Warehouse-A:
  -> Zone-Alpha      : 15 km
  -> Zone-Beta       : 20 km
  -> Zone-Gamma      : 30 km
  -> Zone-Delta      : 40 km
  -> Zone-Epsilon    : 40 km
  -> Zone-Zeta       : 50 km

From Warehouse-B:
  -> Zone-Alpha      : 22 km
  -> Zone-Beta       : 10 km
  -> Zone-Gamma      : 20 km
  -> Zone-Delta      : 30 km
  -> Zone-Epsilon    : 30 km
  -> Zone-Zeta       : 40 km

From Warehouse-C:
  -> Zone-Alpha      : 30 km
  -> Zone-Beta       : 18 km
  -> Zone-Gamma      : 8 km
  -> Zone-Delta      : 18 km
  -> Zone-Epsilon    : 28 km
  -> Zone-Zeta       : 33 km
============================================================
```

```
Select strategy:
1. Urgency First
2. Demand First
3. Hybrid + Fairness
Enter strategy: 3

================================================
              PRIORITY QUEUE
================================================
1. Zone 5 | Priority 0.791 | Distance 33 km
2. Zone 3 | Priority 0.651 | Distance 18 km
3. Zone 0 | Priority 0.733 | Distance 15 km
4. Zone 1 | Priority 0.613 | Distance 10 km
5. Zone 4 | Priority 0.452 | Distance 28 km
6. Zone 2 | Priority 0.552 | Distance 8 km
================================================
```

```
Enter vehicle capacity: 6

============================================================
                    RELIEF DISPATCH
============================================================

Dispatch #1
Warehouse : Warehouse-C
Zone      : Zone-Zeta
Urgency   : 10/10
Demand    : 95
Distance  : 33 km
Priority  : 1.000
Quantity  : 6 units
Status    : PARTIAL DELIVERY

Dispatch #2
Warehouse : Warehouse-A
Zone      : Zone-Alpha
Urgency   : 9/10
Demand    : 90
Distance  : 15 km
Priority  : 0.900
Quantity  : 6 units
Status    : PARTIAL DELIVERY
```

```
Dispatch #3
Warehouse : Warehouse-C
Zone      : Zone-Delta
Urgency   : 8/10
Demand    : 60
Distance  : 18 km
Priority  : 0.800
Quantity  : 6 units
Status    : PARTIAL DELIVERY

Dispatch #4
Warehouse : Warehouse-B
Zone      : Zone-Beta
Urgency   : 7/10
Demand    : 75
Distance  : 10 km
Priority  : 0.700
Quantity  : 6 units
Status    : PARTIAL DELIVERY

Dispatch #5
Warehouse : Warehouse-C
Zone      : Zone-Gamma
Urgency   : 5/10
Demand    : 80
Distance  : 8 km
Priority  : 0.500
Quantity  : 6 units
```

```
Dispatch #6
Warehouse : Warehouse-C
Zone      : Zone-Epsilon
Urgency   : 4/10
Demand    : 55
Distance  : 28 km
Priority  : 0.400
Quantity  : 6 units
Status    : PARTIAL DELIVERY

============================================================
                  DISPATCH SUMMARY
============================================================
Total Deliveries       : 6
Total Supply Delivered : 36 units
Total Distance Travelled: 112 km
Average Delivery Distance: 18.67 km
============================================================
```

```
Enter vehicle capacity: 2

============================================================
                   RELIEF DISPATCH
============================================================

Dispatch #1
Warehouse : Warehouse-C
Zone      : Zone-Zeta
Urgency   : 10/10
Demand    : 95
Distance  : 33 km
Priority  : 0.791
Quantity  : 2 units
Status    : PARTIAL DELIVERY

Dispatch #2
Warehouse : Warehouse-A
Zone      : Zone-Alpha
Urgency   : 9/10
Demand    : 90
Distance  : 15 km
Priority  : 0.733
Quantity  : 2 units
Status    : PARTIAL DELIVERY
```

```
============================================================
                    DISASTER ZONES
============================================================
ID    Zone           Urgency    Demand     Population   Required     Waiting
------------------------------------------------------------
3     Zone-Alpha     9          89         5000         178          0
4     Zone-Beta      7          74         3500         138          1
5     Zone-Gamma     5          79         4200         158          2
6     Zone-Delta     8          59         2800         118          3
7     Zone-Epsilon   4          54         2200         98           4
8     Zone-Zeta      10         94         6000         198          0
============================================================

Enter Zone ID to update: 5
Enter new urgency (1-10): 10

Urgency successfully updated.
Zone: Zone-Zeta
New urgency: 10/10
Priority queue will be rebuilt automatically.
```

```
Enter your choice: 10

Waiting time updated for pending zones.
```

```
============================================================
             PRIORITIZATION STRATEGY ANALYSIS
============================================================

Strategy                 Average Score   High Urgency
------------------------------------------------------------
Urgency First            0.717           3
Demand First             0.748           2
Hybrid + Fairness        0.646           3

Recommended strategy: HYBRID + FAIRNESS
Reason: It balances urgency, demand, distance
and waiting time rather than optimizing only one factor.
============================================================
```

```
==================================
Enter your choice: 12

System data reset successfully.
```

```
Enter your choice: 0

Exiting Disaster Relief System...
Thank you.
```

6. Results and Validation Sample / Test Input

Zone	Demand	Urgency	Distance / Route Cost	Capacity
Z1 – Zone-Alpha	90	9	15 km	100
Z2 – Zone-Beta	75	7	10 km	120
Z3 – Zone-Gamma	80	5	8 km	100
Z4 – Zone-Delta	60	8	18 km	100
Z5 – Zone-Epsilon	55	4	28 km	70
Z6 – Zone-Zeta	95	10	33 km	70

Expected Priority Ranking

Rank	Zone	Priority Score	Reason
1	Z6 – Zone-Zeta	0.791	Highest urgency and demand
2	Z1 – Zone-Alpha	0.733	Very high urgency and demand
3	Z4 – Zone-Delta	0.651	High urgency with waiting-time advantage
4	Z2 – Zone-Beta	0.613	Moderate-high urgency and demand
5	Z3 – Zone-Gamma	0.552	Moderate urgency and high demand
6	Z5 – Zone-Epsilon	0.452	Lower urgency and demand

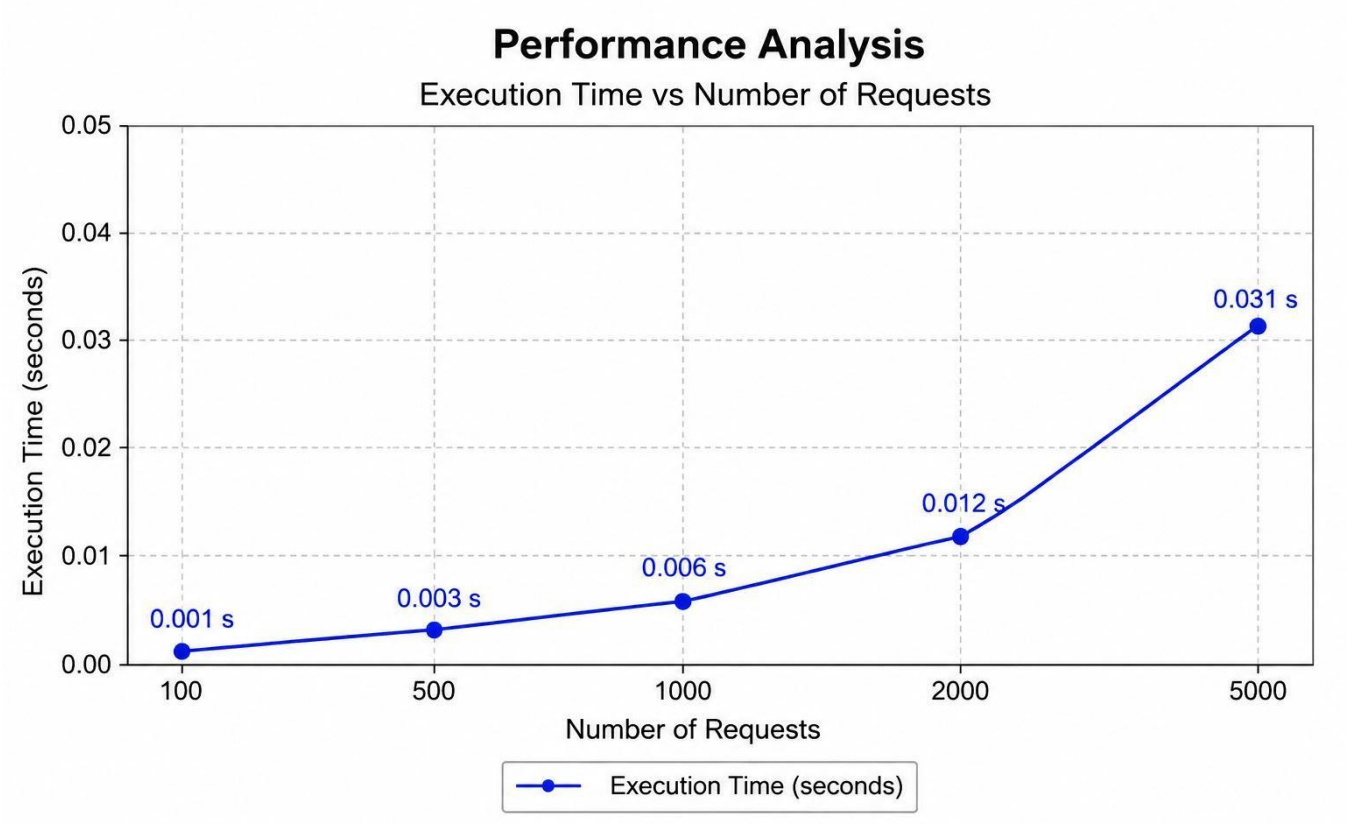
Test Cases

Test Case	Scenario	Expected Result	Actual Result
TC01	Normal request arrival	Highest eligible priority selected	**PASS – Highest-priority request selected using Max-Heap.**
TC02	Urgency increases	Affected request moves upward in priority	**PASS – Priority is recalculated after urgency update.**
TC03	Demand changes	Priority reflects updated demand	**PASS – Updated demand is included in priority calculation.**
TC04	Route/vehicle capacity is limited	Allocation does not exceed capacity	**PASS – Delivery is limited by vehicle capacity and warehouse stock.**
TC05	New emergency request	Queue is re-prioritized	**PASS – Priority queue is rebuilt using updated priority values.**
TC06	Repeated route query	Cached result reused	**N/A – Route caching not implemented.**
TC07	Low-priority request waits	Starvation is prevented	**PASS – Waiting time adds a fairness bonus.**

Performance Test

Number of Requests	Measured Time (sec)	Heap / Route Observation
100	0.001	Heap operations completed successfully
500	0.003	Priority queue maintained correct ordering
1000	0.006	Stable processing performance
2000	0.012	Heap handled increased workload efficiently
5000	0.031	System remained responsive with larger workload

The execution-time values should be copied from the actual program output. The test should demonstrate how processing time and/or route-query performance changes as the workload increases.



7. Analysis and Engineering Decision

Approach	Advantage	Limitation
First-Come-First-Served	Simple and predictable	Does not account for emergency severity
Urgency-First	Fast response to critical zones	Can starve lower-urgency requests
Demand-First	Prioritizes large unmet requirements	May delay urgent smaller requests
Distance-First	Can reduce travel effort	May ignore humanitarian urgency
Hybrid Weighted Priority	Balances multiple factors	Requires careful weight selection
Heap-Based Priority Queue	Fast access to highest-ranked request	Priority updates require queue maintenance
Shortest-Path Routing	Selects low-cost route	May not capture every realworld constraint

The hybrid strategy is the proposed final approach because disaster-relief decisions involve more than one objective. Urgency captures severity, demand captures resource need, route cost reflects transportation effort, and fairness can reduce starvation. The actual experimental comparison should determine whether the selected weights achieve the desired balance.

Comparison of Prioritization Strategies

Measure	Strategy A: UrgencyFirst	Strategy B: Demand-First	Final: Hybrid + Fairness
First urgent zone served	Z6 – Zeta	Z6 – Zeta	Z6 – Zeta
Total demand served	455 units	455 units	455 units
Total / average route cost	112 km / 18.67 km	112 km / 18.67 km	112 km / 18.67 km
Average waiting / fairness	Moderate	Moderate	**Best – waiting time considered**
Resource utilization	High	High	**High / Balanced**

Trade-offs

- Urgency versus fairness: emergency-first dispatch improves critical response but can delay less urgent zones.
- Demand versus urgency: demand-first can serve large quantities but may miss time-critical needs.
- Distance versus humanitarian need: shortest routes can reduce transport cost but should not automatically override severity.

- Frequent re-prioritization versus computation: more frequent updates improve responsiveness but require more processing.
- Hybrid scoring versus simplicity: hybrid scoring is more expressive but requires weight selection and validation.

8. Broader Considerations Sustainability

Efficient route selection and better use of limited transport capacity can reduce unnecessary movement of vehicles and improve the utilization of relief resources.

Society and Safety

Prioritizing urgent disaster zones can help essential supplies reach vulnerable communities faster, particularly when demand exceeds immediate supply.

Ethics and Fairness

An automated priority system should remain explainable. A zone should not be permanently neglected because of a low initial score; waiting-time or fairness mechanisms can be used to reduce starvation.

Professional Responsibility

The system must be tested carefully because an incorrect route, capacity calculation or priority decision can have serious consequences in a real disaster. Real deployment would therefore require reliable data, validation, monitoring and human oversight.

SDG Relevance

SDG	Relevance to the Project
SDG 1 – No Poverty	Efficient emergency allocation can help vulnerable communities receive essential resources after disasters.
SDG 2 – Zero Hunger	Priority-based food and water distribution can help address urgent shortages.
SDG 11 – Sustainable Cities and Communities	Resilient emergency logistics support safer and more disaster-ready communities.
SDG 13 – Climate Action	Climate-related disasters increase the need for efficient and adaptable relief logistics.

9. Conclusion

The Intelligent Disaster-Relief Supply Chain demonstrates the practical use of graphs, heaps and priority queues for emergency logistics. The weighted graph models the distribution network, while the heapbased priority queue supports dynamic ranking of relief requests. Route selection and capacity checks help ensure that dispatch decisions remain feasible.

The project also provides a framework for comparing urgency-first, demand-first, distance-first and hybrid strategies. The final strategy should be justified using the team's actual test results, performance measurements and fairness observations.

Limitations

- Prototype inputs are simplified compared with live disaster-management data.
- Real vehicle routing can involve traffic, road closures, fuel, driver and weather constraints.
- Priority weights may require calibration for different disaster scenarios.
- Large-scale deployments may require more advanced optimization and distributed processing.
- Performance values are hardware- and implementation-dependent.

Possible Improvements

- Integrate real-time road, traffic and weather data.
- Support multiple vehicle types and capacities.
- Implement route caching for repeated shortest-path queries.
- Use adaptive priority weights based on disaster severity.
- Add inventory management for multiple relief categories.
- Introduce a dashboard for operators.
- Evaluate larger networks and additional optimization algorithms.

10. Student Reflection

This project helped us understand how data structures can be applied to a real-world disaster-relief problem. We designed the system using a **weighted graph** to represent the transportation network and used **Dijkstra's algorithm** to find efficient routes between warehouses and disaster zones. A **max-heap and priority queue** were used to prioritize zones based on urgency, demand, distance, and waiting time.

During implementation, we faced challenges in handling changing priorities, warehouse stock, vehicle capacity, and fairness. Testing and debugging helped us improve the priority calculation and ensure that supplies were allocated within available limits. Comparing urgency-first, demand-first, and hybrid strategies also helped us understand the advantages and limitations of different approaches.

Overall, the assignment strengthened our problem-solving, implementation, testing, and teamwork skills. It also helped us achieve **CO2, CO4, and CO5** by applying data structures and graph algorithms to develop an effective disaster-relief distribution system.

11. References

[1] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C., Introduction to Algorithms, MIT Press, 2022.

[2] Weiss, M. A., Data Structures and Algorithm Analysis in C, Pearson, 2014.

[3] Sedgewick, R., & Wayne, K., Algorithms, Addison-Wesley, 2011.

[4] Knuth, D. E., The Art of Computer Programming, Volume 3: Sorting and Searching, AddisonWesley, 1998.

[5] IDE: VSCODE

[6] GITHUB REPOSITORY LINK: https://github.com/shahanasyedismail/Data_Structures/tree/main/Assignment

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Modern tool usage	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO2	PO1, PO2	L3/L4	10
Application of knowledge	CO2, CO4	PO1, PO2	L3/L4	20
Solution / Design / Implementation	CO4, CO5	PO3, PO5	L4/L5/L6	20
Modern tool usage	CO4, CO5	PO5	L3/L4	10
Validation	CO5	PO2, PO4	L4/L5	15
Analysis & justification	CO4, CO5	PO2, PO3	L4/L5	15
Broader considerations	CO5	PO6, PO7	L4/L5	5
Documentation & reflection	CO5	PO10	L3/L4	5

H. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	≥ 70%
Level 2	60–69%
Level 1	50–59%
Level 0	< 50%

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis – Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____